

Cell 1 — Install Required Libraries

```
!pip install networkx matplotlib pandas ipywidgets
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Requirement already satisfied: notebook>=4.4.1 in /usr/local/lib/python3.12/dist-packages (from widgetsnbextension~=3.6.0->ipywidgets) (4.4.1)
Requirement already satisfied: parso<0.9.0,>=0.8.6 in /usr/local/lib/python3.12/dist-packages (from jedi>=0.16->ipython>=4.0.0) (0.8.6)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->ipykernel) (0.4)
Requirement already satisfied: jupyter-core>=4.9.2 in /usr/local/lib/python3.12/dist-packages (from jupyter-client>=6.1.12->ipykernel) (4.9.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (3.1.2)
Requirement already satisfied: argon2-cffi in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (25.1.0)
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (5.10.4)
Requirement already satisfied: nbconvert>=5 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (7.12.2)
Requirement already satisfied: Send2Trash>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (1.8.2)
Requirement already satisfied: terminado>=0.8.3 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.17.0)
Requirement already satisfied: prometheus-client in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.20.0)
Requirement already satisfied: nbclassic>=0.4.7 in /usr/local/lib/python3.12/dist-packages (from notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.4.7)
Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.12/dist-packages (from pexpect>4.3->ipython>=4.0.0->ipykernel) (0.7.0)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from prompt-toolkit!=3.0.0,<3.1.0,>=3.0.1->ipython>=4.0.0->ipykernel) (0.2.13)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.9.2->jupyter-client) (4.2.0)
Requirement already satisfied: notebook-shim>=0.2.3 in /usr/local/lib/python3.12/dist-packages (from nbclassic>=0.4.7->notebook) (0.2.3)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (4.12.3)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (6.1.0)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.7.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.2.2)
Requirement already satisfied: markupsafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (2.1.5)
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (3.0.2)
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.8.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (1.5.1)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat->notebook>=4.4.1->widgetsnbextension~=3.6.0) (2.20.1)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat->notebook>=4.4.1->widgetsnbextension~=3.6.0) (4.19.0)
Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.12/dist-packages (from argon2-cffi->notebook>=4.4.1->widgetsnbextension~=3.6.0) (21.2.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.11)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (1.2.0)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->notebook>=4.4.1->widgetsnbextension~=3.6.0) (23.2.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->notebook>=4.4.1->widgetsnbextension~=3.6.0) (2023.12.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.35.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.18.0)
Requirement already satisfied: jupyter-server<3,>=1.8 in /usr/local/lib/python3.12/dist-packages (from notebook-shim>=0.2.3->notebook>=4.4.1->widgetsnbextension~=3.6.0) (1.24.0)
Requirement already satisfied: cffi>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from argon2-cffi-bindings->argon2-cffi) (1.17.1)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (2.5)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~=3.6.0) (4.11.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi) (2.22)
Requirement already satisfied: anyio>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook>=4.4.1->widgetsnbextension~=3.6.0) (4.6.2)
Requirement already satisfied: jupyter-events>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.10.0)
Requirement already satisfied: jupyter-server-terminals>=0.4.4 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook>=4.4.1->widgetsnbextension~=3.6.0) (0.4.4)
Requirement already satisfied: overrides>=5.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook>=4.4.1->widgetsnbextension~=3.6.0) (7.7.0)
Requirement already satisfied: websocket-client>=1.7 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook>=4.4.1->widgetsnbextension~=3.6.0) (1.7.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio>=3.1.0->jupyter-server<3,>=1.8->notebook>=4.4.1->widgetsnbextension~=3.6.0) (3.10.1)
Requirement already satisfied: python-json-logger>=2.0.4 in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server) (2.0.4)
Requirement already satisfied: pyyaml>=5.3 in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server) (6.0.1)
Requirement already satisfied: rfc3339-validator in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server) (0.1.1)
Requirement already satisfied: rfc3986-validator>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server) (0.1.1)
Requirement already satisfied: fqdn in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-server) (1.5.1)
Requirement already satisfied: isoduration in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-server) (2.0.0)
Requirement already satisfied: jsonpointer>1.13 in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-server) (2.3)
Requirement already satisfied: rfc3987-syntax>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-server) (1.1.0)
Requirement already satisfied: uri-template in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-server) (0.0.1)
Requirement already satisfied: webcolors>=24.6.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-server) (24.6.0)
Requirement already satisfied: lark>=1.2.2 in /usr/local/lib/python3.12/dist-packages (from rfc3987-syntax>=1.1.0->jsonschema[format-nongpl]>=4.18.0->jupyter-server) (1.2.2)
Requirement already satisfied: arrow>=0.15.0 in /usr/local/lib/python3.12/dist-packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-server) (1.3.0)
```

Cell 2 — Imports

```
import io
import base64
import time
import heapq
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

from collections import deque
from IPython.display import display, HTML, clear_output
import ipywidgets as widgets
```

CELL 3 — Graph Setup

```

G = nx.Graph()

nodes = {
    "Attacker": 10,
    "Firewall": 8,
    "Honeypot": 3,
    "DMZ": 7,
    "Web": 6,
    "Mail": 6,
    "Internal": 5,
    "Auth": 3,
    "DB": 2,
    "Target": 0
}

for n, h in nodes.items():
    G.add_node(n, h=h)

edges = [
    ("Attacker", "Firewall", 3),
    ("Attacker", "Honeypot", 1),
    ("Firewall", "DMZ", 2),
    ("DMZ", "Web", 1),
    ("DMZ", "Mail", 2),
    ("Web", "Internal", 2),
    ("Mail", "Internal", 3),
    ("Internal", "DB", 4),
    ("Internal", "Auth", 2),
    ("DB", "Target", 2),
    ("Auth", "Target", 4)
]

for u, v, w in edges:
    G.add_edge(u, v, weight=w)

def h(n):
    return G.nodes[n]["h"]

```

Cell 4 — Layout

```

pos = {
    "Attacker": (-4, 0),
    "Honeypot": (-2, -1.8),
    "Firewall": (-2.5, 1.8),
    "DMZ": (-1.2, 0.8),
    "Web": (0.3, 1.8),
    "Mail": (0.3, -0.2),
    "Internal": (1.8, 0.8),
    "Auth": (3, -1.2),
    "DB": (3, 1.8),
    "Target": (4.5, 0.2)
}

```

CELL 5 — Algorithms

```

from collections import deque
import heapq

# =====
# BFS
# =====
def bfs(s, g):
    q = deque([[s]])
    vis = set()
    expanded = 0

    while q:
        p = q.popleft()
        n = p[-1]

        if n in vis:

```

```

        continue

    vis.add(n)
    expanded += 1

    if n == g:
        return p, expanded

    for i in G[n]:
        if i not in vis:
            q.append(p + [i])

    return [], expanded

# =====
# DFS
# =====
def dfs(s, g):
    st = [[s]]
    vis = set()
    expanded = 0

    while st:
        p = st.pop()
        n = p[-1]

        if n in vis:
            continue

        vis.add(n)
        expanded += 1

        if n == g:
            return p, expanded

        for i in G[n]:
            if i not in vis:
                st.append(p + [i])

    return [], expanded

# =====
# UCS
# =====
def ucs(s, g):
    pq = [(0, [s])]
    vis = set()
    best_cost = {s: 0}
    expanded = 0

    while pq:
        c, p = heapq.heappop(pq)
        n = p[-1]

        if n in vis:
            continue

        vis.add(n)
        expanded += 1

        if n == g:
            return p, expanded

        for i in G[n]:
            new_cost = c + G[n][i]['weight']

            if i not in best_cost or new_cost < best_cost[i]:
                best_cost[i] = new_cost
                heapq.heappush(pq, (new_cost, p + [i]))

    return [], expanded

# =====
# A* SEARCH

```

```
# =====
def astar(s, g):
    pq = [(h(s), 0, [s])] # (f = g+h, g, path)
    vis = set()
    best_cost = {s: 0}
    expanded = 0

    while pq:
        f, c, path = heapq.heappop(pq)
        n = path[-1]

        if n in vis:
            continue

        vis.add(n)
        expanded += 1

        if n == g:
            return path, expanded

        for i in G[n]:
            new_cost = c + G[n][i]['weight']

            if i not in best_cost or new_cost < best_cost[i]:
                best_cost[i] = new_cost
                f_score = new_cost + h(i)
                heapq.heappush(pq, (f_score, new_cost, path + [i]))

    return [], expanded

# =====
# HILL CLIMBING
# =====
def hill(s, g):
    current = s
    path = [s]
    expanded = 0

    while current != g:
        expanded += 1
        neighbors = list(G[current])

        if not neighbors:
            break

        # choose best heuristic neighbor
        next_node = min(neighbors, key=lambda x: h(x))

        if h(next_node) >= h(current):
            break # stuck in local optimum

        current = next_node
        path.append(current)

    return path, expanded
```

Cell 6 — Minimax + Alpha Beta Pruning

```
def alphabeta(node, depth, alpha, beta, maximizing):
    if depth == 0 or node == "Target":
        return -h(node)

    if maximizing:
        best = -999

        for n in G[node]:
            val = alphabeta(n, depth-1, alpha, beta, False)
            best = max(best, val)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best
```

```

else:
    best = 999

    for n in G[node]:
        val = alphabeta(n, depth-1, alpha, beta, True)
        best = min(best, val)
        beta = min(beta, best)

        if beta <= alpha:
            break

    return best

def minimax_path(start, depth=4):
    path = [start]
    cur = start
    expanded = 0

    for _ in range(depth):
        if cur == "Target":
            break

        best_node = None
        best_val = -999

        for n in G[cur]:
            expanded += 1
            val = alphabeta(n, 3, -999, 999, False)

            if val > best_val:
                best_val = val
                best_node = n

        if best_node is None:
            break

        cur = best_node
        path.append(cur)

    return path, expanded

```

CELL 7 — Cost Function

```

def cost(path):
    if len(path) < 2:
        return 0

    c = 0
    for u, v in zip(path[:-1], path[1:]):
        c += G[u][v]["weight"]

    return c

```

CELL 8 — Graph Image Generator

```

def draw(path=[]):
    plt.figure(figsize=(6,4))

    node_colors = []

    for n in G.nodes():
        if n == "Attacker":
            node_colors.append("cyan")
        elif n == "Target":
            node_colors.append("red")
        elif n in path:
            node_colors.append("yellow")
        else:
            node_colors.append("gray")

    nx.draw(G, pos, with_labels=True, node_color=node_colors, node_size=1200)
    labels = nx.get_edge_attributes(G, 'weight')
    nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

```

```
buf = io.BytesIO()
plt.savefig(buf, format='png')
buf.seek(0)

img = base64.b64encode(buf.read()).decode("utf-8")
plt.close()

return img
```

Cell 9 — html and css

```
HTML_STYLE = r"""
.ios-container {
  background-color: #000000;
  color: #ffffff;

  font-family: -apple-system, BlinkMacSystemFont, "SF Pro Display", "Segoe UI", sans-serif;

  padding: 24px;
  border-radius: 20px;
  box-shadow: 0 15px 35px rgba(0,0,0,0.8);
  margin: 10px auto;
  max-width: 980px;
  letter-spacing: -0.2px;
}

.dash-header {
  margin-bottom: 20px;
  border-bottom: 1px solid #1c1c1e;
  padding-bottom: 15px;
  position: relative;
}

.dash-header h1 {
  font-size: 24px;
  font-weight: 700;
  margin: 0;

  background: linear-gradient(180deg, #ffffff 0%, #a1a1a6 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;

  display: inline-block;
}

.subtitle {
  color: #8e8e93;
  font-size: 11px;
  margin: 4px 0 0 0;
  text-transform: uppercase;
  letter-spacing: 1px;
}

.live-indicator {
  width: 8px;
  height: 8px;
  background-color: #30d158;
  border-radius: 50%;
  display: inline-block;
  box-shadow: 0 0 8px #30d158;
  margin-right: 8px;
  vertical-align: middle;
}

.main-layout-grid {
  display: grid;
  grid-template-columns: 1.2fr 1fr;
  gap: 20px;
}

@media (max-width: 768px) {
  .main-layout-grid {
    grid-template-columns: 1fr;
  }
}
```

```
.visual-card,
.control-panel-card {
  background: rgba(28, 28, 30, 0.8);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);

  border: 1px solid rgba(255,255,255,0.06);
  border-radius: 16px;
  padding: 18px;
}

.visual-card h3 {
  font-size: 14px;
  font-weight: 600;
  color: #8e8e93;
  margin: 0 0 12px 0;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.img-container {
  background: #1c1c1e;
  border-radius: 12px;
  overflow: hidden;

  display: flex;
  align-items: center;
  justify-content: center;

  border: 1px solid rgba(255,255,255,0.03);
}

.responsive-img {
  max-width: 100%;
  height: auto;
  display: block;
}

.telemetry-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 12px;
}

.telemetry-header h2 {
  font-size: 20px;
  font-weight: 700;
  margin: 0;
}

.badge {
  background: rgba(10, 132, 255, 0.15);
  color: #0a84ff;
  padding: 3px 8px;
  border-radius: 6px;
  font-size: 11px;
  font-weight: 600;
}

.description {
  color: #c7c7cc;
  font-size: 13px;
  line-height: 1.4;
  margin: 0 0 18px 0;
}

.stat-row {
  display: flex;
  gap: 10px;
  margin-bottom: 18px;
}

.stat-tile {
  flex: 1;
  background: #1c1c1e;
```

```

border: 1px solid rgba(255,255,255,0.04);
border-radius: 10px;
padding: 10px;
text-align: center;
}

.stat-lbl {
color: #8e8e93;
font-size: 10px;
text-transform: uppercase;
font-weight: 600;
display: block;
margin-bottom: 4px;
}

.stat-val {
font-size: 14px;
font-weight: 700;
}

.highlight-green { color: #30d158; }
.highlight-purple { color: #bf5af2; }
.highlight-blue { color: #0a84ff; }

.path-result-box {
background: #1c1c1e;
border-radius: 12px;
padding: 14px;
border: 1px solid rgba(255,255,255,0.04);
}

.rendered-path {
display: flex;
flex-wrap: wrap;
gap: 6px;
align-items: center;
}

.node-tag {
padding: 2px 6px;
border-radius: 5px;
font-size: 10px;
font-weight: 700;
color: #ffffff;
background-color: #3a3a3c;
}

.node-tag.attacker { background-color: #0a84ff; }
.node-tag.target { background-color: #ff453a; }
.node-tag.firewall { background-color: #ff9f0a; }
.node-tag.dnz { background-color: #bf5af2; }
.node-tag.internal { background-color: #64d2ff; color: #000; }
.node-tag.db { background-color: #ac8e68; }
.node-tag.honeypot { background-color: #af52de; }
.node-tag.web { background-color: #30d158; color: #000; }
.node-tag.mail { background-color: #5856d6; }
.node-tag.auth { background-color: #ff3b30; }
"""

```

CELL 10 — MAIN RENDER ENGINE

```

output = widgets.Output()

def render(change=None):
    selected = dropdown.value

    start, goal = "Attacker", "Target"
    t0 = time.time()

    if selected == "BFS":
        path, expanded = bfs(start, goal)
        strategy, desc = "Uninformed Search", "BFS exploration"

    elif selected == "DFS":
        path, expanded = dfs(start, goal)

```



```

        strategy, desc = "Uninformed Search", "DFS deep search"

    elif selected == "UCS":
        path, expanded = ucs(start, goal)
        strategy, desc = "Cost Search", "Uniform Cost Search"

    elif selected == "A*":
        path, expanded = astar(start, goal)
        strategy, desc = "Heuristic Search", "A* search"

    elif selected == "Hill Climbing":
        path, expanded = hill(start, goal)
        strategy, desc = "Greedy", "Local search"

    elif selected == "Minimax":
        path, expanded = minimax_path(start)
        strategy, desc = "Game AI", "Minimax + Alpha Beta"

    else:
        df = compare_all()
        with output:
            clear_output()
            display(df)
        return

    latency = (time.time() - t0) * 1000
    graph_img = draw(path)

    path_html = " → ".join(
        [f'<span class="node-tag {n.lower()}">{n}</span>' for n in path]
    )

    html = f"""
<div class="ios-container">
  <div class="dash-header">
    <span class="live-indicator"></span>
    <h1>Threat Simulation Terminal</h1>
    <p class="subtitle">Live Algorithm Execution Frame</p>
  </div>

  <div class="main-layout-grid">

    <div class="visual-card">
      <h3>Dynamic Path Visualization</h3>
      <div class="img-container">
        
      </div>
    </div>

    <div class="control-panel-card">

      <div class="telemetry-header">
        <span class="badge">{strategy}</span>
        <h2>{selected}</h2>
      </div>

      <p class="description">{desc}</p>

      <div class="stat-row">
        <div class="stat-tile">
          <span class="stat-lbl">Latency</span>
          <span class="stat-val highlight-green">{latency:.3f}</span>
        </div>

        <div class="stat-tile">
          <span class="stat-lbl">Cost</span>
          <span class="stat-val highlight-purple">{cost(path)}</span>
        </div>

        <div class="stat-tile">
          <span class="stat-lbl">Nodes</span>
          <span class="stat-val highlight-blue">{expanded}</span>
        </div>
      </div>

      <div class="path-result-box">
        <div class="rendered-path">{path_html}</div>

```

```

        </div>

    </div>

</div>
</div>

<style>
{HTML_STYLE}
</style>
"""

with output:
    clear_output()
    display(HTML(html))

```

CELL 11 — COMPARE ALL MATRIX

```

def compare_all():
    algos = {
        "BFS": bfs,
        "DFS": dfs,
        "UCS": ucs,
        "A*": astar,
        "Hill Climbing": hill,
        "Minimax": minimax_path
    }

    rows = []

    for name, func in algos.items():
        t0 = time.time()

        if name == "Minimax":
            p, exp = func("Attacker")
        else:
            p, exp = func("Attacker", "Target")

        t = (time.time() - t0) * 1000

        rows.append([
            name,
            len(p),
            cost(p),
            exp,
            round(t, 3)
        ])

    return pd.DataFrame(rows, columns=[
        "Algorithm", "Path Length", "Cost", "Nodes", "Time(ms)"
    ])

```

CELL 12 — UI

```

dropdown = widgets.Dropdown(
    options=["BFS", "DFS", "UCS", "A*", "Hill Climbing", "Minimax", "Compare All"],
    value="BFS",
    description="Algorithm:"
)

dropdown.observe(render, names="value")

display(widgets.VBox([dropdown, output]))

render()

```

Algorithm: Hill Climbing

LIVE ALGORITHM EXECUTION FRAME

DYNAMIC PATH VISUALIZATION

